ASSIGNMENT 2-CO2

NAME:Tharani RB
REG NO:1911260138
COURCE CODE:CSA0822
SLOT:B
GITHUB: GitHub - TharaniRB15/CSA0822-1911260138-

TASK 1-Decisions with Many Boundaries

AIM:

To write a Python program to determine electricity subsidy eligibility based on household electricity consumption and age using separate if statements and if-elif-else statements.

THEORY:

Decision-making statements in Python are used to execute different blocks of code based on conditions.

1. If statement: Executes a block when the condition is true.
2. If-elif-else: Checks multiple conditions and executes only one matching block.
3. Boolean operators: and, or are used to combine conditions.
4. Chained comparisons: Used to check a range of values, such as 101 <= units <= 200.

PROGRAM:

File Edit Format Run Options Window Help

```
units = int(input("Enter electricity units: "))
age = int(input("Enter age of household member: "))

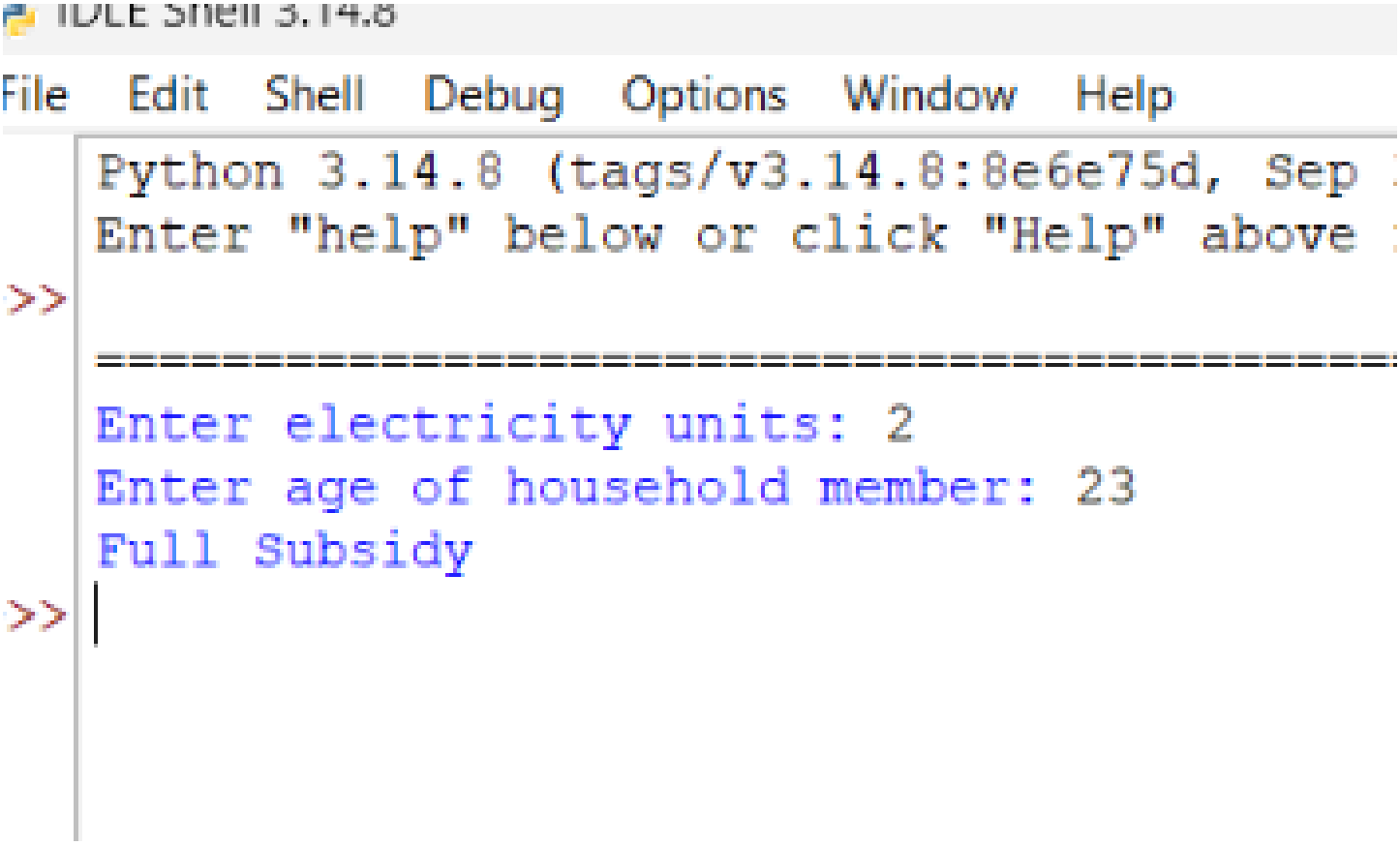
if units <= 100:
    print("Full Subsidy")

if 101 <= units <= 200 and age >= 18:
    print("Partial Subsidy")

if units > 200 and age <= 80:
    print("No Subsidy")

if units > 200 and age > 80:
    print("Special Subsidy")
```

OUTPUT:



TASK 2 -Loops that Count Attempts

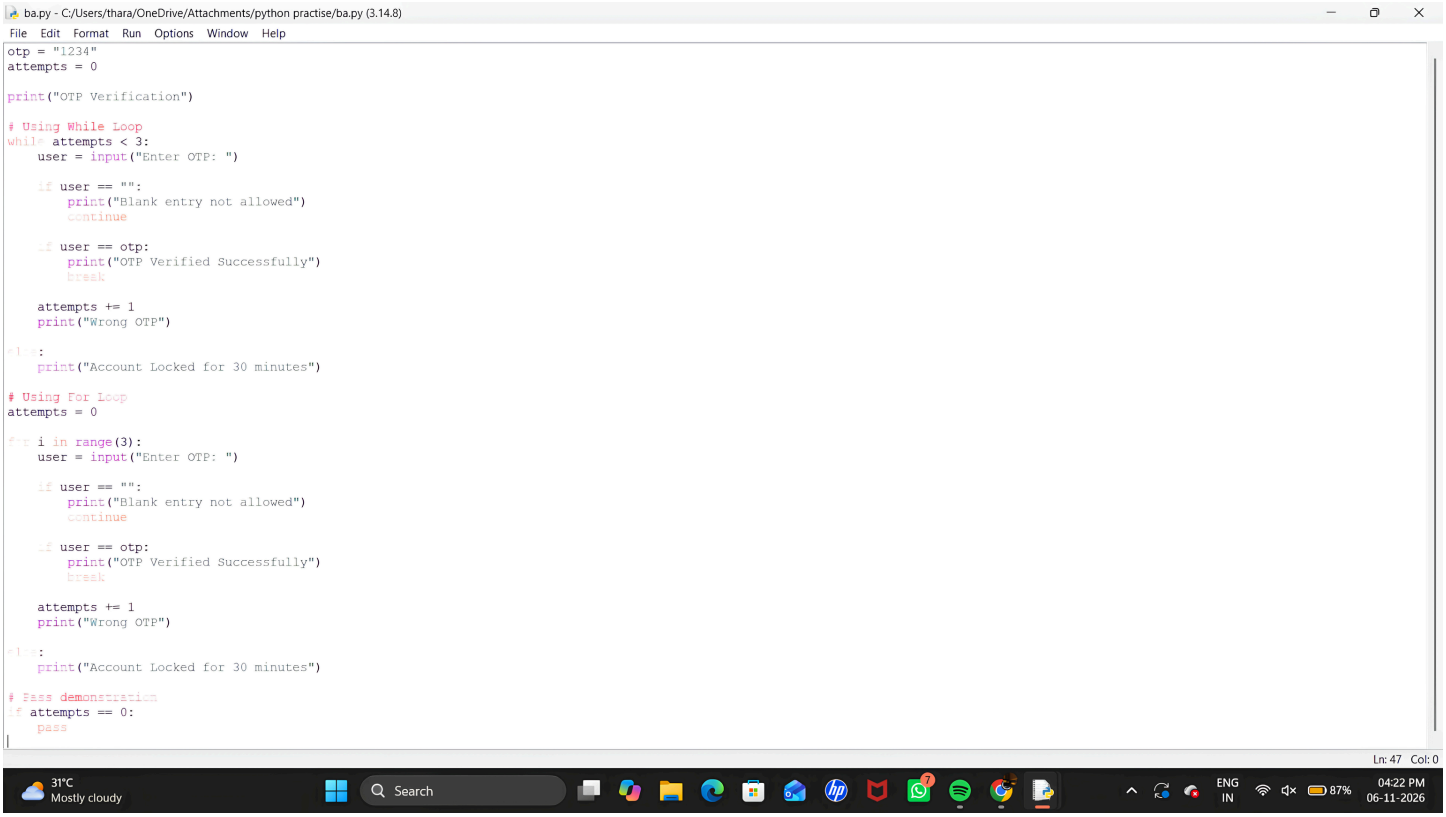
AIM:

To write a Python program to verify OTP using while and for loops, allowing three attempts and locking the account after three incorrect attempts.

THEORY:

- While Loop: Repeats a block of code until the condition becomes false.
- For Loop: Repeats code for a fixed number of times using range().
- Break: Stops the loop immediately.
- Continue: Skips the current iteration and moves to the next iteration.
- Pass: Acts as an empty statement without performing any action.
- Loop Else: Executes when the loop finishes without using break.

PROGRAM:



```
otp = "1234"
attempts = 0

print("OTP Verification")

# Using While Loop
while attempts < 3:
    user = input("Enter OTP: ")

    if user == "":
        print("Blank entry not allowed")
        continue

    if user == otp:
        print("OTP Verified Successfully")
        break

    attempts += 1
    print("Wrong OTP")

else:
    print("Account Locked for 30 minutes")

# Using For Loop
attempts = 0

for i in range(3):
    user = input("Enter OTP: ")

    if user == "":
        print("Blank entry not allowed")
        continue

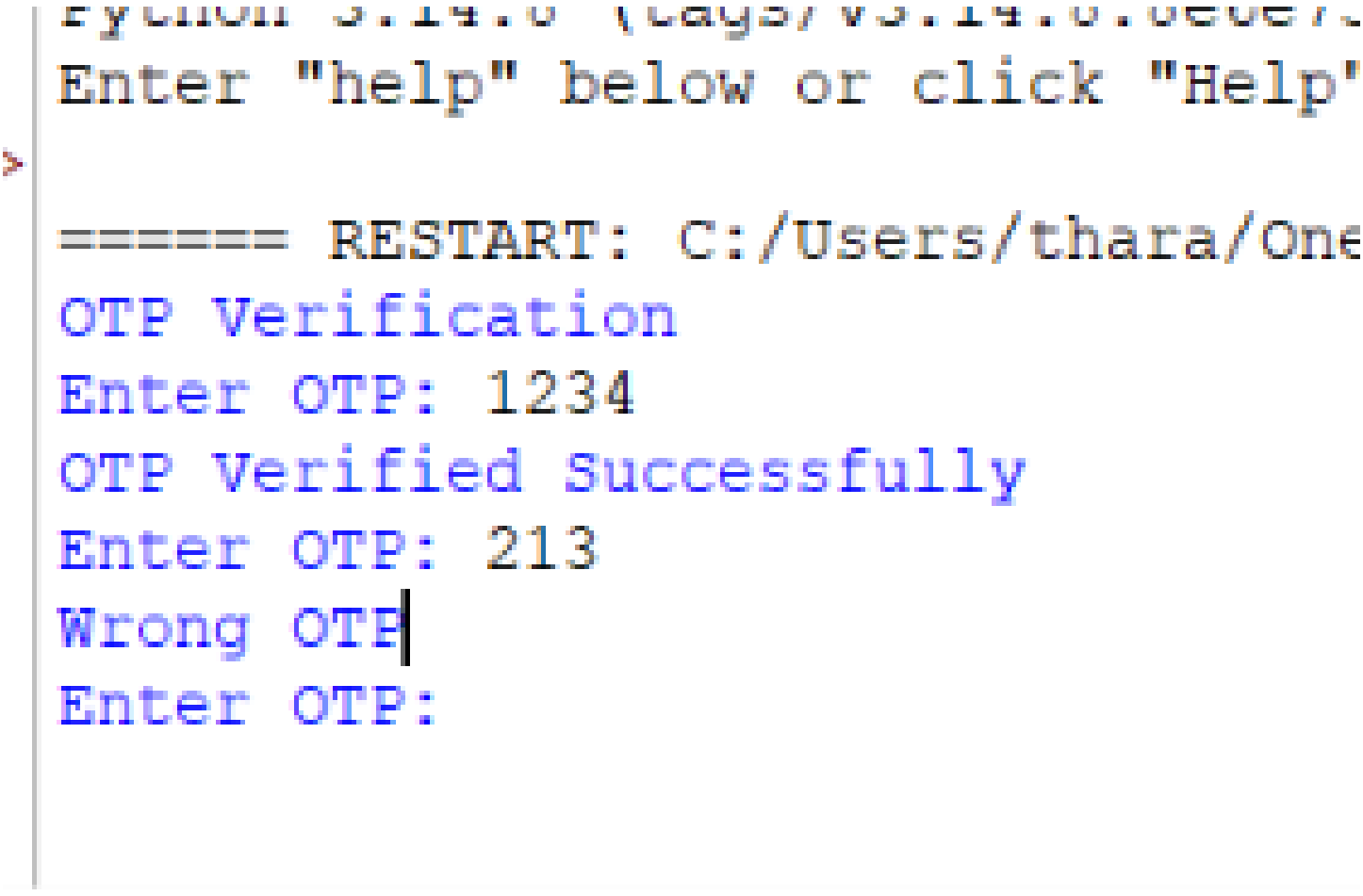
    if user == otp:
        print("OTP Verified Successfully")
        break

    attempts += 1
    print("Wrong OTP")

else:
    print("Account Locked for 30 minutes")

# Pass demonstration
if attempts == 0:
    pass
```

OUTPUT:



```
Enter "help" below or click "Help"

>

====== RESTART: C:/Users/thara/One
OTP Verification
Enter OTP: 1234
OTP Verified Successfully
Enter OTP: 213
Wrong OTP
Enter OTP:
```

TASK-3 Engineer a Vehicle Registration Number Validator

AIM:

To write a Python program to normalize, validate, and format vehicle registration numbers using string operations and slicing without regular expressions

THEORY:

- String Methods: Used to modify and check strings.
- Upper(): Converts lowercase letters into uppercase.
- Replace(): Removes spaces and hyphens from a string.
- Strip(): Removes unwanted spaces from the beginning and end.
- Slicing: Extracts specific characters from a string.
- Isalpha(): Checks whether characters contain only alphabets.
- Isdigit(): Checks whether characters contain only digits.
- Len(): Finds the length of a string.

PROGRAM:

ba.py - C:/Users/thara/OneDrive/Attachments/python practise/ba.py (3.14.8)

Edit Format Run Options Window Help

```
f normalise(text):
  text = text.strip().upper()
  text = text.replace(" ", "")
  text = text.replace("-", "")
  return text
f is_valid(reg):
  if len(reg) not in (9, 10):
      return False
  if not reg[0:2].isalpha():
      return False
  if not reg[2:4].isdigit():
      return False
  if len(reg) == 10:
      if not reg[4:6].isalpha():
          return False
      if not reg[6:10].isdigit():
          return False
  elif len(reg) == 9:
      if not reg[4:5].isalpha():
          return False
      if not reg[5:9].isdigit():
          return False
  return True
f format_number(reg):
  if len(reg) == 9:
      return reg[0:2] + " " + reg[2:4] + " " + reg[4:5] + " " + reg[5:9]
  else:
      return reg[0:2] + " " + reg[2:4] + " " + reg[4:6] + " " + reg[6:10]
r raw in ["tn 09 ab 1234", "KA-01-M-0007",
          "TN09AB1234", "TN 22 1234 AB",
          "MH 12 CD 5678", "TN 9 AB 1234"]:
  reg = normalise(raw)
  if is_valid(reg):
      print(raw, " -> ", reg, " Valid")
      print("Receipt:", format_number(reg))
  else:
      print(raw, " -> ", reg, " INVALID")
```

OUTPUT:

```
>>>
====== RESTART: C:/Users/thara/OneDrive/
tn 09 ab 1234  ->  TN09AB1234  Valid
Receipt: TN 09 AB 1234
KA-01-M-0007  ->  KA01M0007  Valid
Receipt: KA 01 M 0007
TN09AB1234  ->  TN09AB1234  Valid
Receipt: TN 09 AB 1234
TN 22 1234 AB  ->  TN221234AB  INVALID
MH 12 CD 5678  ->  MH12CD5678  Valid
Receipt: MH 12 CD 5678
TN 9 AB 1234  ->  TN9AB1234  INVALID
>>>
```

TASK-4 – Building Long Text: + or join()

AIM:

To write a Python program to build a large SMS bill statement using string concatenation (+=) and join() methods and compare their execution times.

THEORY:

- String Concatenation (+): Combines two or more strings into a single string.
- Join(): Combines multiple strings into one string using a separator.
- String Immutability: Strings cannot be changed after creation. Every modification creates a new string.
- Time Complexity: Measures how much time a program takes to execute.
- time.perf_counter(): Measures the execution time of a program accurately.

- List: Stores multiple string items that can be combined using join()

PROGRAM:

File Edit Format Run Options Window Help

```
import time
items = [f"Item{i}: Rs.{i % 50}" for i in range(200000)]
def build_plus(parts):
    text = ""
    for p in parts:
        text += p + ","
    return text
def build_join(parts):
    text = ",".join(parts)
    return text
start = time.perf_counter()
result1 = build_plus(items)
end = time.perf_counter()
plus_time = end - start
start = time.perf_counter()
result2 = build_join(items)
end = time.perf_counter()
join_time = end - start
print("Using += :", plus_time, "seconds")
print("Using join():", join_time, "seconds")
print("\nStatements are equal:", result1.replace(",", "") == result2.replace(",", ""))
```

OUTPUT:

```
Python 3.14.8 (tags/v3.14.8:8e6e75d, Sep 30
Enter "help" below or click "Help" above for

======= RESTART: C:/Users/thara/OneDrive/Atta
Using += : 0.5166678000241518 seconds
Using join(): 0.004046600020956248 seconds

Statements are equal: True
```

TASK 5-Functions, Parameters and Scope

AIM:

To write a Python program using functions, parameters, and scope to manage canteen orders for different customers and avoid sharing order lists between customers

THEORY:

- Function: A reusable block of code that performs a specific task.
- Positional Parameters: Values are passed according to their position.
- Keyword Parameters: Values are passed using parameter names.
- Default Parameters: Provide a default value when no argument is given.
- Local Variable: A variable declared inside a function.

PROGRAM:

File Edit Format Run Options Window Help

```
def add_item_buggy(item, order=[]):
    order.append(item)
    return order
def add_item(item, order=None):
    if order is None:
        order = []
    order.append(item)
    return order
print("Buggy Function:")
print(add_item_buggy("Tea"))
print(add_item_buggy("Vada"))
print("Default Values:", add_item_buggy.__defaults__)
print("\nCorrect Function:")
print(add_item("Tea"))
print(add_item("Vada"))
print("\nCustomer Orders:")
customer1 = add_item("Tea")
customer2 = add_item("Coffee")
customer3 = add_item("Vada")
print("Customer 1:", customer1)
print("Customer 2:", customer2)
print("Customer 3:", customer3)
def calculate_bill(amount):
    gst = amount * 0.05
    total = amount + gst
    return total, gst
total, gst = calculate_bill(100)
print("\nTotal Bill:", total)
print("GST:", gst)
x = 10
def scope_example():
    x = 20
    print("Local x:", x)
scope_example()
print("Global x:", x)
```

```
def outer():
    x = 10
    def inner():
        nonlocal x
        x = 30
    inner()
    print("Nonlocal x:", x)
outer()
```

OUTPUT:

```
====== RESTART: C:/Users/thara/OneD
Buggy Function:
['Tea']
['Tea', 'Vada']
Default Values: (['Tea', 'Vada'],)

Correct Function:
['Tea']
['Vada']

Customer Orders:
Customer 1: ['Tea']
Customer 2: ['Coffee']
Customer 3: ['Vada']

Total Bill: 105.0
GST: 5.0
Local x: 20
Global x: 10
Nonlocal x: 30
```

TASK 20- Recursion and Its Limits

AIM:

To write a Python program to calculate the number of ways to climb a staircase using recursion, memoisation, and loops, and compare their execution efficiency.

THEORY:

- Recursion: A function calling itself to solve a problem.
- Base Case: The condition that stops the recursive function.
- Recursive Case: The function calls itself with smaller values.
- Memoisation: Stores previously calculated results to avoid repeated calculations.
- LRU Cache: Uses @lru_cache to store and reuse function results.

PROGRAM:

```
File  Edit  Format  Run  Options  Window  Help
from functools import lru_cache
import sys
calls = 0
def ways_naive(n):
    global calls
    calls += 1
    if n <= 1:
        return 1
    return ways_naive(n - 1) + ways_naive(n - 2)
@lru_cache(maxsize=None)
def ways_memo(n):
    global calls
    calls += 1
    if n <= 1:
        return 1
    return ways_memo(n - 1) + ways_memo(n - 2)
def ways_loop(n):
    a, b = 1, 1
    for i in range(2, n + 1):
        a, b = b, a + b
    return b
print("Answers from 1 to 6:")
for n in range(1, 7):
    print(n, "steps:", ways_loop(n))
for n in [10, 20, 25, 30]:
    calls = 0
    result1 = ways_naive(n)
    naive_calls = calls
    calls = 0
    result2 = ways_memo(n)
    memo_calls = calls
    print("\nSteps:", n)
    print("Naive:", result1, "Calls:", naive_calls)
    print("Memo:", result2, "Calls:", memo_calls)
sys.setrecursionlimit(2000)
print("\nWays for 1000 steps:")
try:
    calls = 0
    print("Memoisation:", ways_memo(1000))
    print("Function Calls:", calls)
```

OUTPUT:

```
Enter "help" below or click "Help" above

======= RESTART: C:/Users/thara/OneDrive/
Answers from 1 to 6:
1 steps: 1
2 steps: 2
3 steps: 3
4 steps: 5
5 steps: 8
6 steps: 13

Steps: 10
Naive: 89 Calls: 177
Memo: 89 Calls: 11

Steps: 20
Naive: 10946 Calls: 21891
Memo: 10946 Calls: 10

Steps: 25
Naive: 121393 Calls: 242785
Memo: 121393 Calls: 5

Steps: 30
Naive: 1346269 Calls: 2692537
Memo: 1346269 Calls: 5
```

TASK 7-Diagnose, Redesign and Validate

AIM:

To identify the performance limitation of normal recursion and redesign the staircase climbing program using iteration to reduce repeated calculations and improve execution time.

THEORY:

- Bottleneck: The part of a program that takes more time or resources.
- Recursion: A function calling itself to solve a problem.
- Repeated Calculation: Normal recursion calculates the same values multiple times.
- Iteration: Uses loops to solve problems without repeated recursive calls.
- Time Complexity: Measures how execution time increases with input size.

PROGRAM:

```python
import time
calls = 0
def ways_naive(n):
    global calls
    calls += 1
    if n <= 1:
        return 1
    return ways_naive(n-1) + ways_naive(n-2)
def ways_loop(n):
    a, b = 1, 1
    count = 0
    for i in range(2, n+1):
        a, b = b, a+b
        count += 1
    return b, count
for n in [10, 20, 25, 30]:
    calls = 0
    start = time.perf_counter()
    original = ways_naive(n)
    original_time = time.perf_counter() - start
    start = time.perf_counter()
    redesigned, iterations = ways_loop(n)
    redesigned_time = time.perf_counter() - start
    print("\nSteps:", n)
    print("Original Result:", original)
    print("Original Calls:", calls)
    print("Original Time:", original_time)
    print("Redesigned Result:", redesigned)
    print("Loop Iterations:", iterations)
    print("Redesigned Time:", redesigned_time)
    print("Results Equal:", original == redesigned
result, count = ways_loop(1000)
print("\n1000 Steps:")
print("Number of Ways:", result)
print("Loop Iterations:", count)
```

OUTPUT:

```
Steps: 10
Original Result: 89
Original Calls: 177
Original Time: 5.1799986977130175e-05
Redesigned Result: 89
Loop Iterations: 9
Redesigned Time: 8.400005754083395e-06
Results Equal: True

Steps: 20
Original Result: 10946
Original Calls: 21891
Original Time: 0.005149899981915951
Redesigned Result: 10946
Loop Iterations: 19
Redesigned Time: 1.0200019460171461e-05
Results Equal: True

Steps: 25
Original Result: 121393
Original Calls: 242785
Original Time: 0.04528769996250048
Redesigned Result: 121393
Loop Iterations: 24
Redesigned Time: 1.0400020983070135e-05
Results Equal: True

Steps: 30
Original Result: 1346269
Original Calls: 2692537
Original Time: 0.4897546000429429
Redesigned Result: 1346269
Loop Iterations: 29
Redesigned Time: 1.2700038496404886e-05
Results Equal: True
```